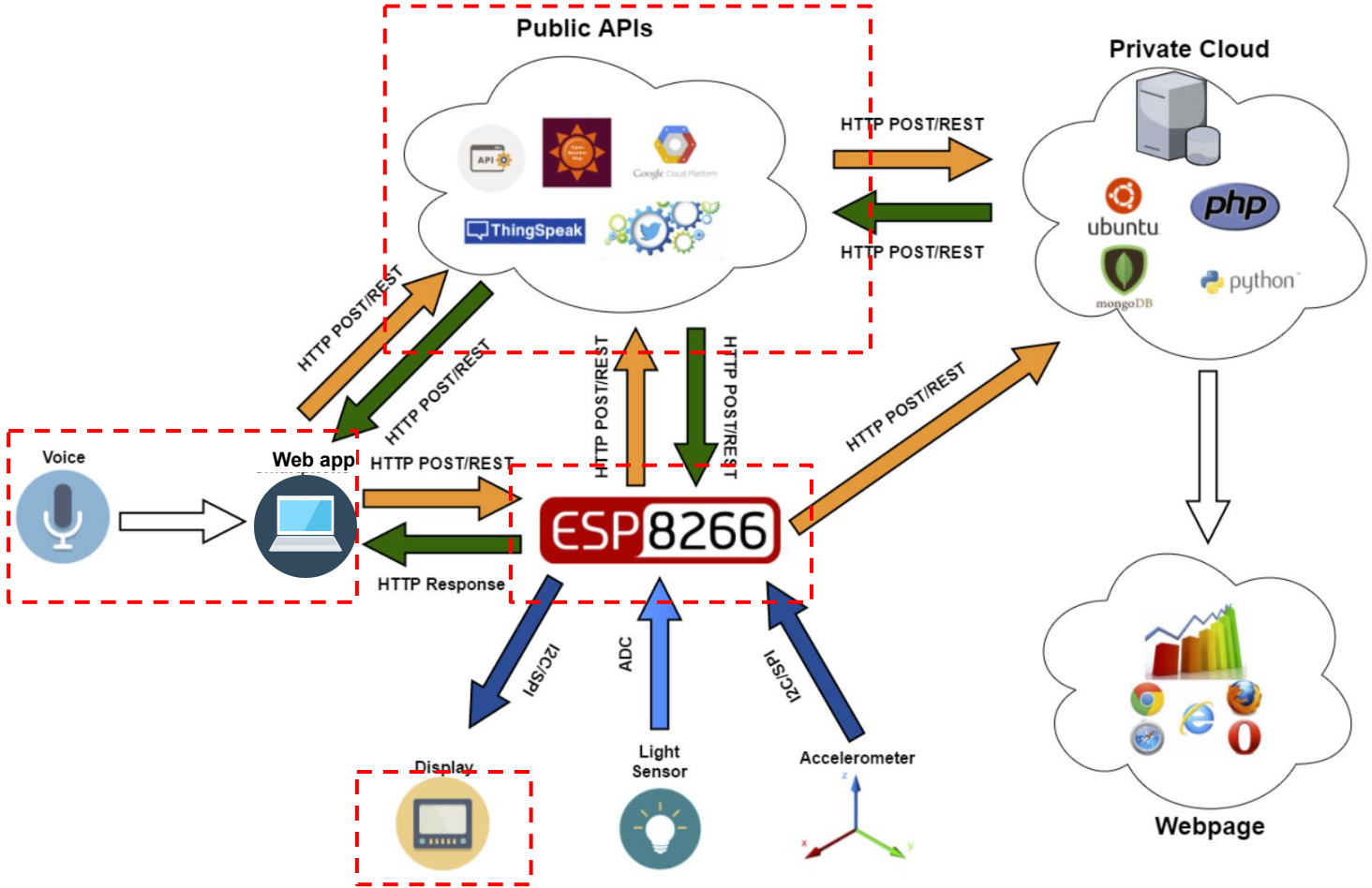
AIoT - Intelligent and Connected Systems
EECS E4764 Fall' 26

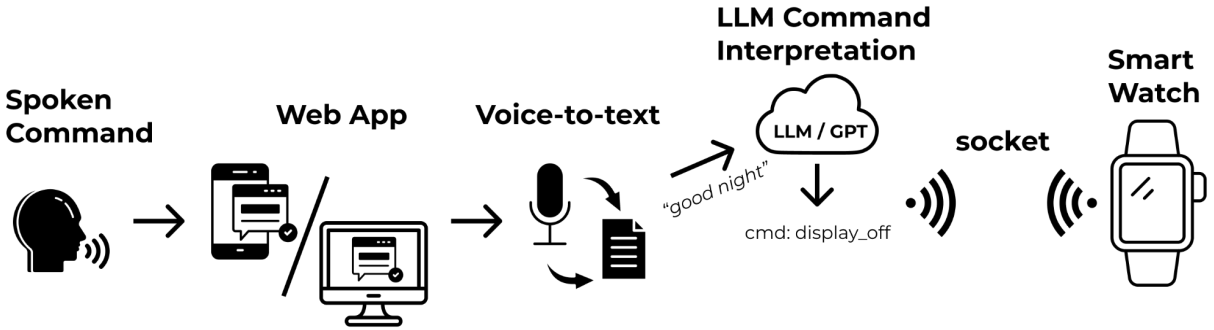
Lab 5

Lab Overview



Lab Overview

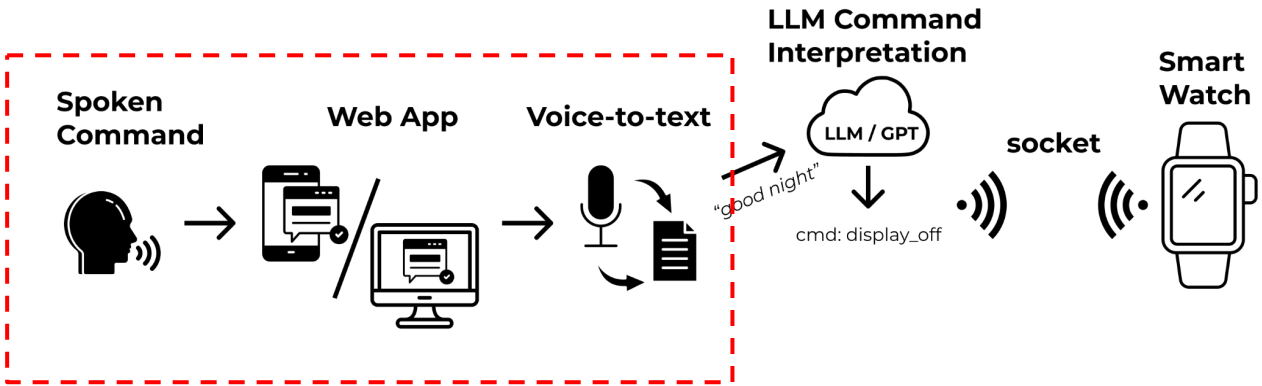
Voice-controlled smartwatch assistant



- Speak a command to the web app
- Recognize command text from the audio
- Convert the command into structured form through LLMs
- Send the command to ESP and operate the specific function

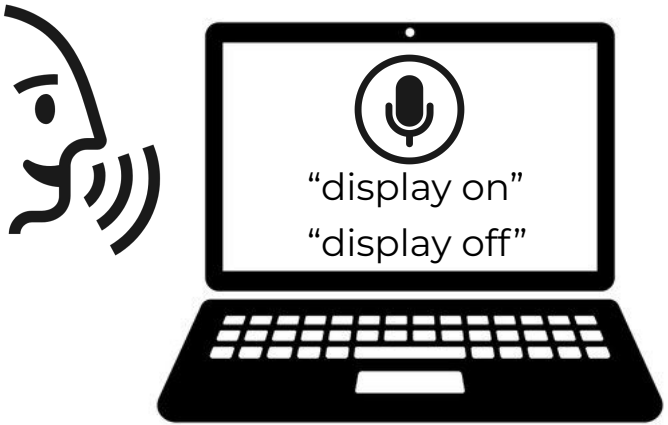
Part 1

User Interface and Voice-to-text

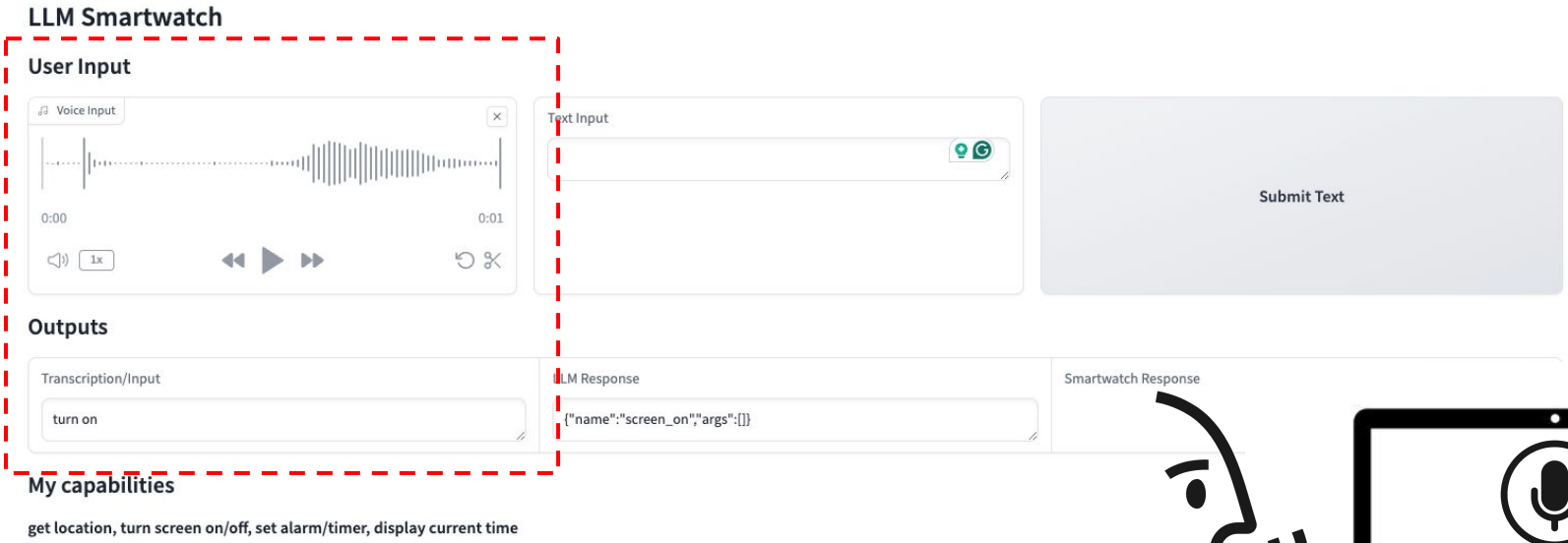


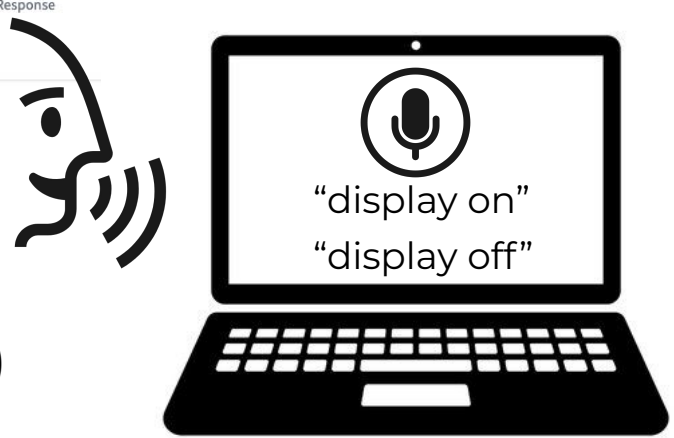
Part 1

- Create a Python-based web application using Python + **Gradio** to record audio
- Integrate speech-to-text to our web app using the open-source **Whisper** speech recognition by OpenAI
- No experience? No problem.
    - Examples and tutorials everywhere



Part 1

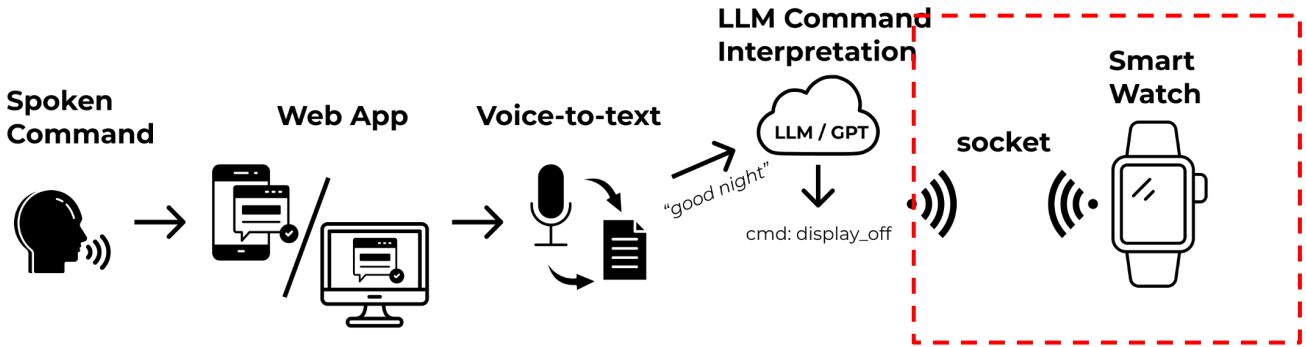




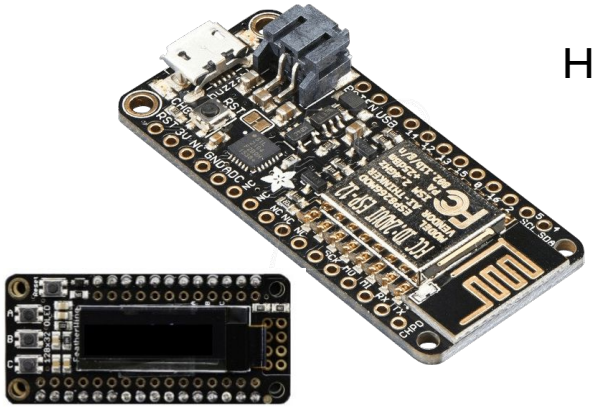
- Record the audio through Web App (**Gradio**) and translate it into text (**Whisper**)

Part 2

Smartwatch Interface and Communication

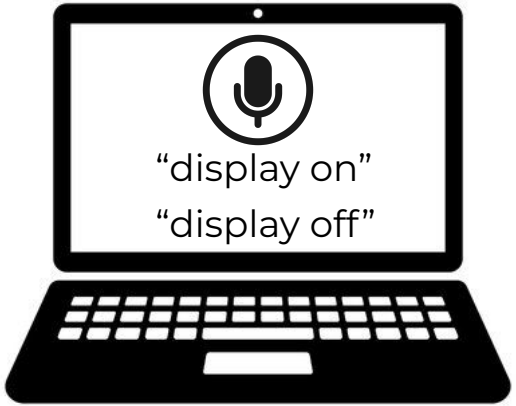


Part 2 Overview - Command



How to match user commands with specific function?

“display on”



Various Functions:

- “Display on/off”
- “Display time”
- “Get location”
- “Display weather”
- “Set alarm”
- “Display messages”

Various Commands

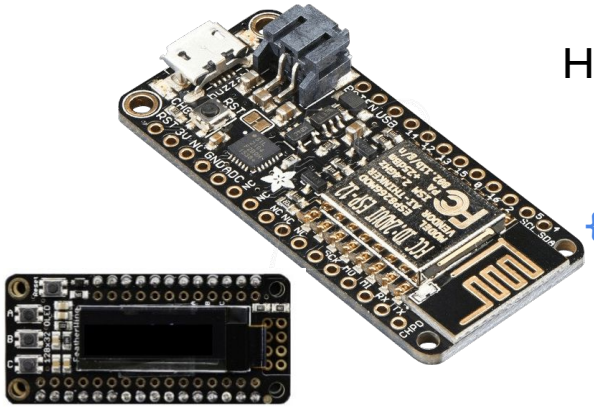
“Display on”

“Set alarm on 9 a.m.”

“Display off”



Part 2 Overview - Command

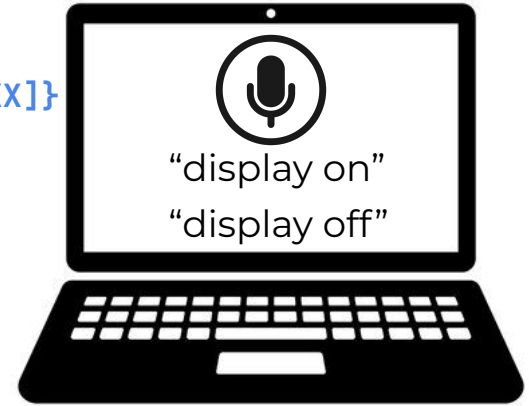


How to match user commands with specific function?

Standardize Your commands!

`{name: <function_name>, args: [XX, XX]}`

“display on”



Various Functions:

- “Display on/off”
- “Display time”
- “Get location”
- “Display weather”
- “Set alarm”
- “Display messages”

`{name: "screen_on", args: []}`

`{name: set_alarm, args: [hour(24 hour format), minute, second]}`

Various Commands

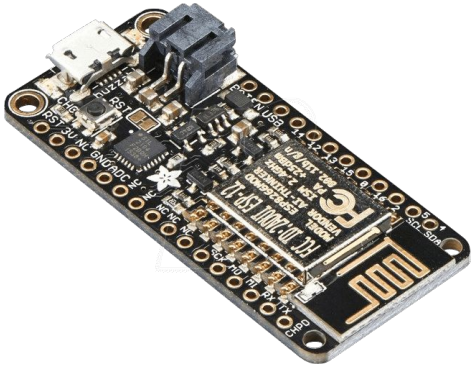
“Display on”

“Set alarm on 9 a.m.”

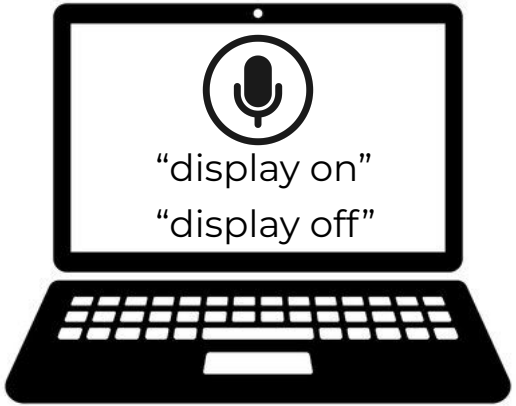
“Display off”

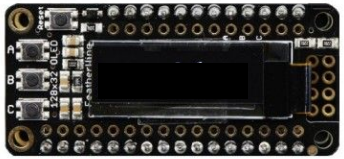


Part 2 Overview - Server

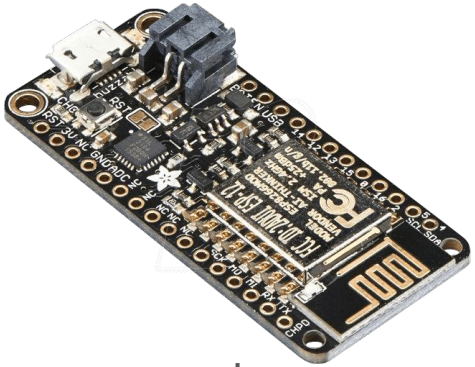


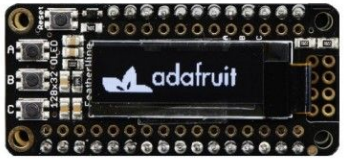
“display on” ->
`{name: "screen_on", args: []}`





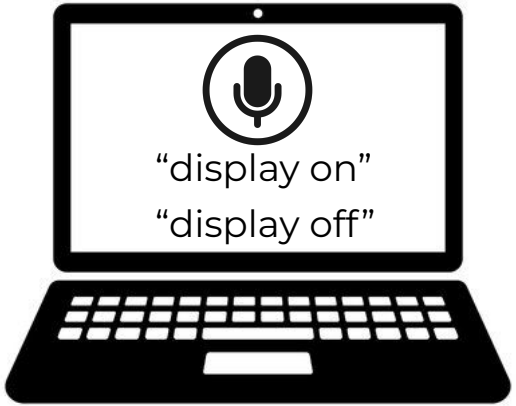
Part 2 Overview - Server



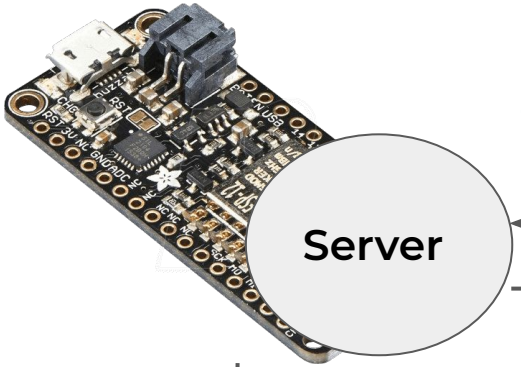


“display on” ->
`{name: "screen_on", args: []}`

OK! Display turned on.



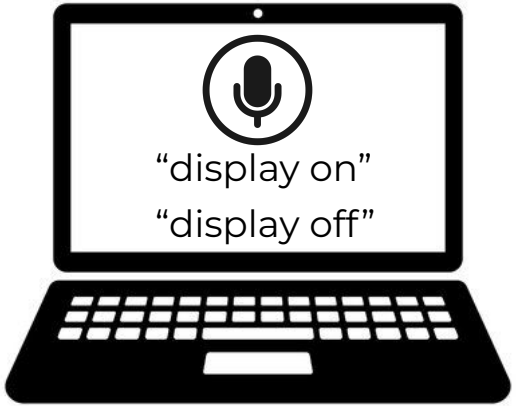
How to listen to commands and respond?

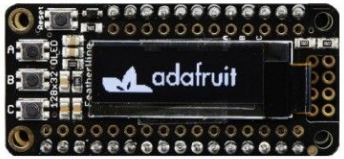


Server

“display on” ->
`{name: "screen_on", args: []}`

OK! Display turned on.





How to build a simple HTTP server?

- Use socket in MicroPython!
- Usage almost same as on computer Python

```
addr = ('0.0.0.0',80)
s = socket.socket()
s.bind(addr)
s.listen(1)
while(1):
    try:
        (conn, address) = s.accept()
    except OSError:
        print("Nothing")
    else:
        rec = conn.recv(4096)
        rec = rec.split(b'\r\n\r\n')
        ............
```

https://docs.micropython.org/en/latest/library/socket.html
https://docs.micropython.org/en/latest/library/socket.html#socket.socket.settimeout
https://docs.micropython.org/en/latest/library/socket.html#socket.socket.setblocking

Testing

- Develop and test code in Python first, before putting into ESP
- Test with a computer client running Python

Testing

- Develop and test code in Python first, before putting into ESP
- Test with a computer client running Python
  - Use ESP's Local IP Address

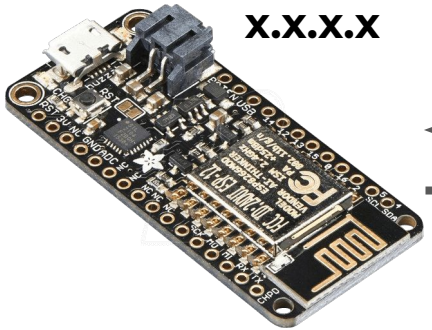
```
wlan = network.WLAN(network.STA_IF)
...
# After wlan.isconnected()
print(wlan.ifconfig())
```

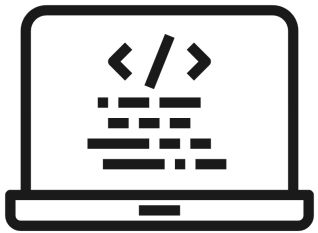
```
IP/netmask/gw/DNS addresses
```

```
addr = ('0.0.0.0',80)
s = socket.socket()
s.bind(addr)
s.listen(1)
while(1):
    try:
        (conn, address) = s.accept()
    except OSError:
        print("Nothing")
    else:
        rec = conn.recv(4096)
        rec = rec.split(b'\r\n\r\n')
        ...........
```

Same Wi-Fi

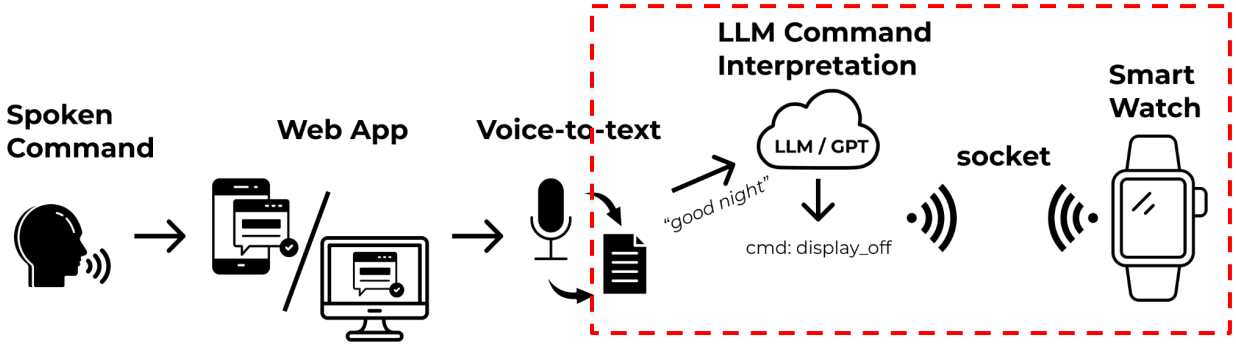
x.x.x.x





Part 3

Natural Language Processing with LLM



Part 3 Overview - LLMs API

Gemini

Generate your own free API key at:

https://aistudio.google.com/app/api-keys

python -m pip install google-genai

Gemini API Sample Code

```python
import asyncio
from google import genai

GEMINI_API_KEY = "Enter-API-KEY-HERE"

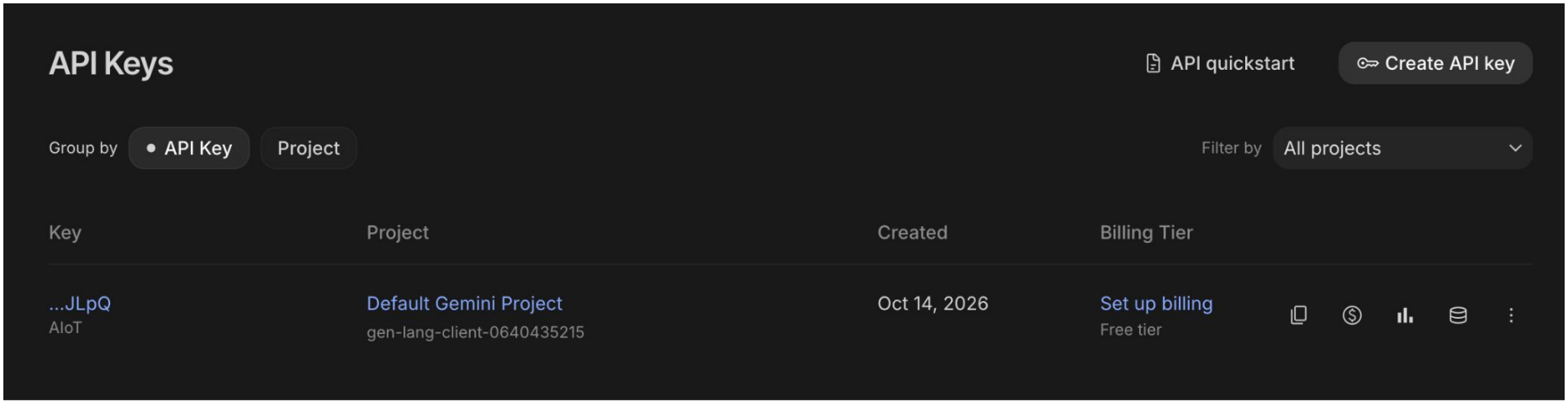
async def get_llm_response(prompt):
    full_response = ""

    async with genai.Client(api_key=GEMINI_API_KEY).aio as client:
        async for partial in await client.models.generate_content_stream(
            model="gemini-3.5-flash-lite",
            contents=prompt,
        ):
            if partial.text:
                full_response += partial.text

    return full_response


if __name__ == "__main__":
    prompt = input("Enter a prompt: ")
    print(asyncio.run(get_llm_response(prompt)))
```

Getting Gemini API Key



Part 3 Overview - LLMs API

Model:

gemini-3.5-flash-lite

(No need to use other models)

Gemini API Sample Code

```python
import asyncio
from google import genai

GEMINI_API_KEY = "Enter-API-KEY-HERE"

async def get_llm_response(prompt):
    full_response = ""

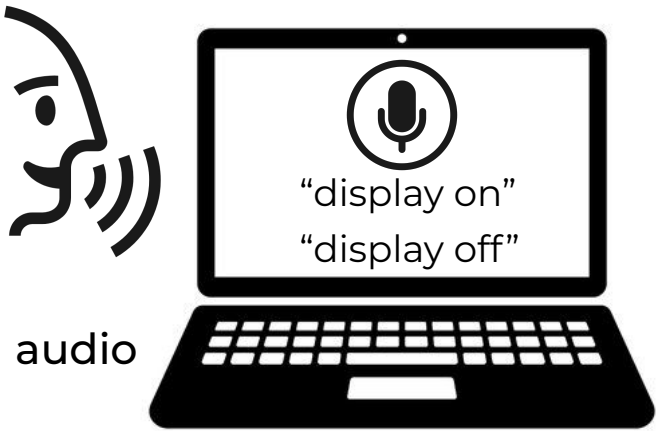
    async with genai.Client(api_key=GEMINI_API_KEY).aio as client:
        async for partial in await client.models.generate_content_stream(
            model="gemini-3.5-flash-lite",
            contents=prompt,
        ):
            if partial.text:
                full_response += partial.text

    return full_response


if __name__ == "__main__":
    prompt = input("Enter a prompt: ")
    print(asyncio.run(get_llm_response(prompt)))
```

Part 3 Overview - LLMs Prompt

- Design a system prompt to convert command texts into corresponding standardized command



audio → **Whisper** → text → **gemini-3 .5-flash** → `{name: <function_name>, args: [XX, XX]}`

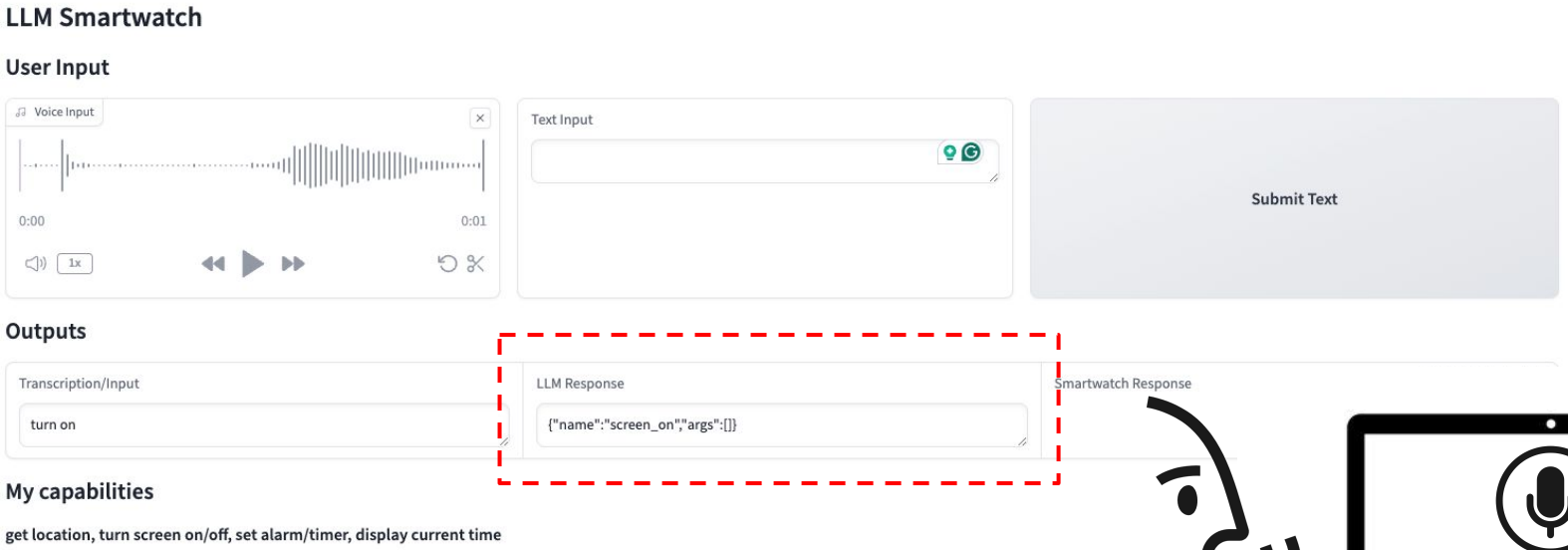
“Display on”

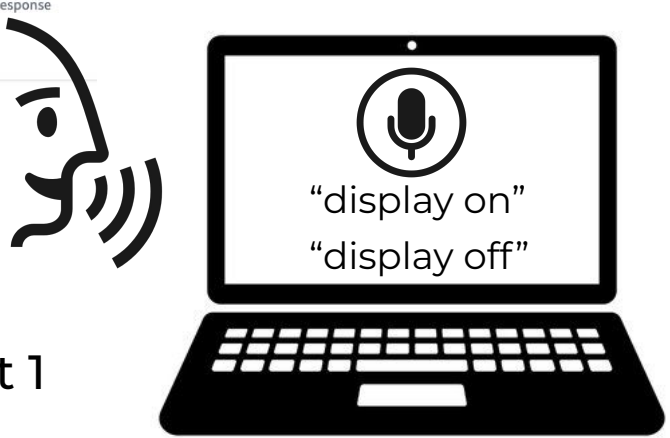
“Turn on the screen”

“Light the screen”

`{name: "screen_on", args: []}`

Part 3

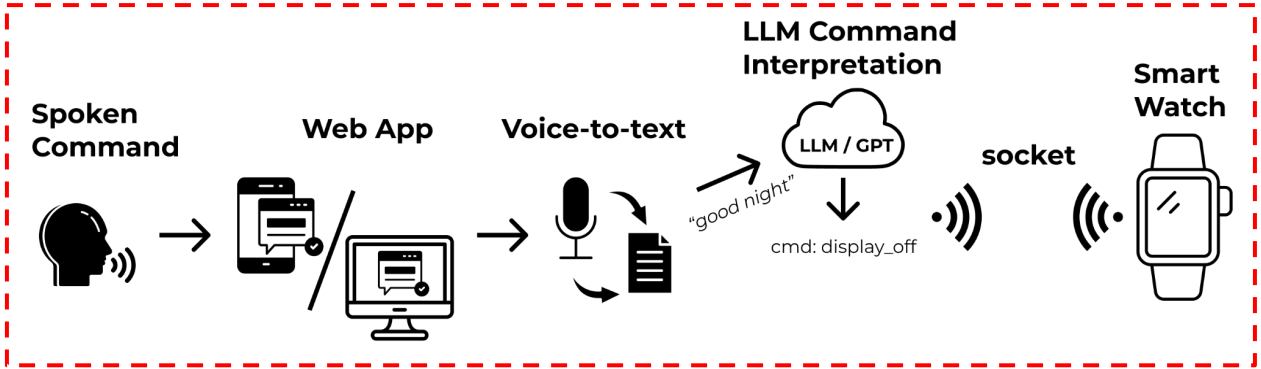




- Connect your LLM processing capabilities with the Gradio interface we created in Part 1

Part 4

Final Integration

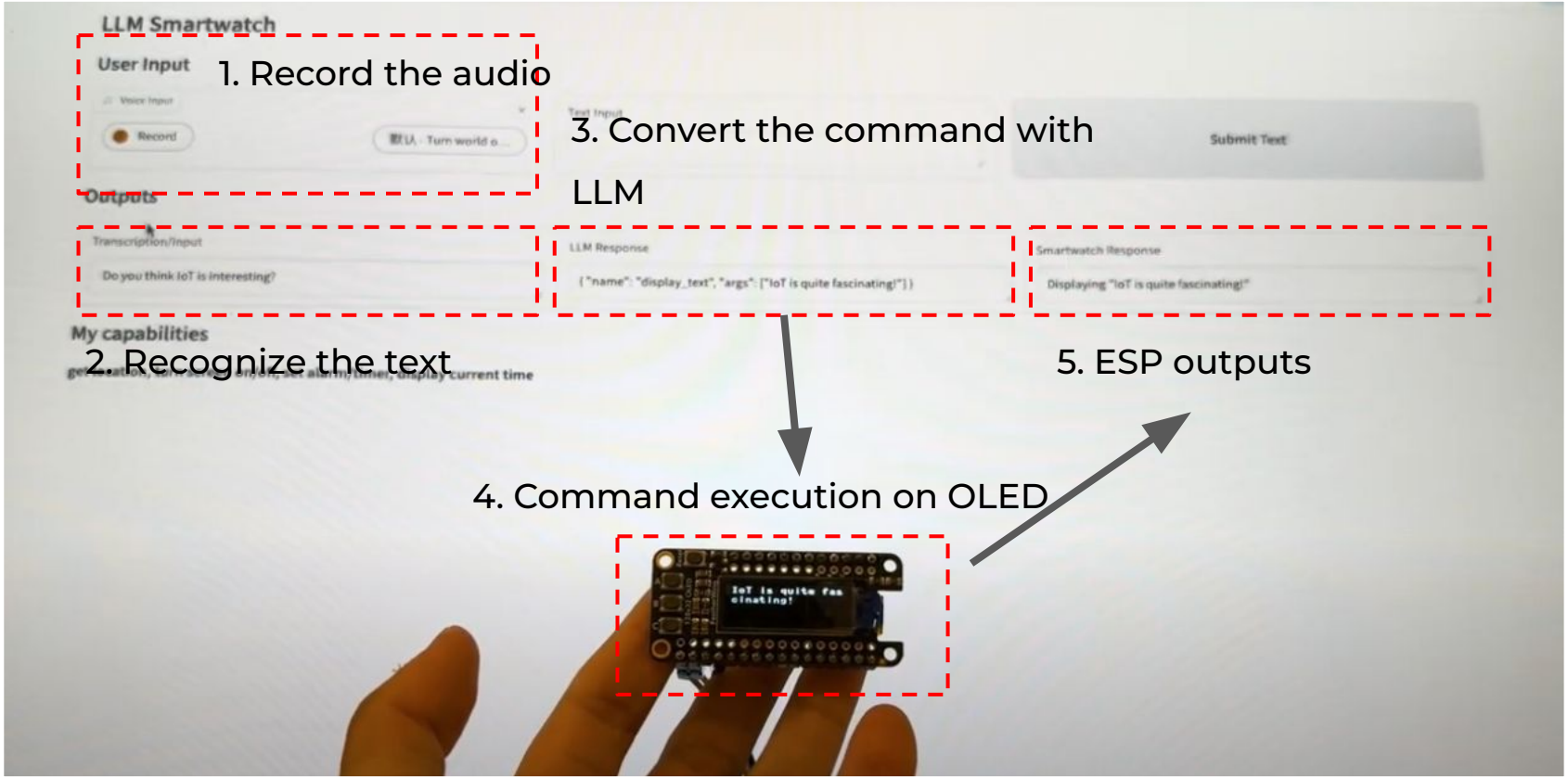


What to expect

- Implement commands:
  - “Display on/off”: turn on or off the OLED display
  - “Display time”: behave like a regular digital watch
  - “Get location”: get your location and display on the OLED
  - “Display weather”: display weather information of current location
  - “Set alarm”: set an alarm that triggers visual (LED) and/or audio (piezzo) notifications at alarm time
  - Other messages: display the message on the OLED display
- Two-way communication
  - ESP sends back a response of command execution status
  - App displays the confirmation message received

Part 4

https://drive.google.com/file/d/13eAfyLkWlpdSC6OpmFEGOWDoQqHmtaJx/view?usp=drive_link



Lab 5 Checkpoints

1. Make a simple web application using gradio that enables voice recognition. Display the text after speaking to the microphone.
2. Demonstrate that you can control various smartwatch functions remotely using the JSON interface we defined.
3. Integrate LLM to accurately interpret a wide range of natural language commands related to smartwatch functions, and consistently generate valid JSON commands to be sent to the smart watch.
4. Demonstrate your fully integrated voice-controlled smartwatch assistant

Logistics

- Due in 2 weeks (Oct. 28)
- Many components can be developed and tested in parallel -- team work!

Part 3 Overview - LLMs API

Poe

Model:

GPT-4o-Mini/GPT-5-Mini

(No need to use other models)

Poe API Sample Code

```python
import asyncio
import fastapi_poe as fp

POE_API_KEY = ''

async def get_llm_response(prompt):
    message = fp.ProtocolMessage(role="user", content=prompt)
    full_response = ""
    async for partial in fp.get_bot_response(messages=[message],
                                             bot_name='GPT-4o-Mini',
                                             api_key=POE_API_KEY):
        full_response += partial.text
    return full_response

if __name__ == "__main__":
    prompt = input("Enter a prompt: ")
    print(asyncio.run(get_llm_response(prompt)))
```